



**HASHDIT**

**HashDit**

Smart Contract Security

---

# ListaDao ListaV3FactoryOwner Security Audit Report

ListaDao

Oct 05 2026 - Oct 06 2026

# Disclaimer

---

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

## Audit Overview

---

|                         |                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| Audit Period            | Oct 05 2026 - Oct 06 2026                                                                               |
| Overall Risk [Optional] | Informational                                                                                           |
| Project Name            | ListaDao                                                                                                |
| Github                  | <a href="https://github.com/lista-dao/lista-v3/pull/3">https://github.com/lista-dao/lista-v3/pull/3</a> |
| Commit                  | audit/v3                                                                                                |

# Smart Contracts List

| No. | File Name               | Verdict       | Details        |
|-----|-------------------------|---------------|----------------|
| 1   | ListaV3FactoryOwner.sol | Informational | [I01]<br>[I02] |

# Findings

## [I01] Hardcoded max collection amounts in collectProtocolFees() reduces flexibility and can strand single-sided pool fees

Contract      ListaV3FactoryOwner.sol

Severity      **Informational**

### Description

ListaV3FactoryOwner.collectProtocolFees() always calls the pool's collectProtocol with both requested amounts set to type(uint128).max. The underlying ListaV3Pool.collectProtocol supports per side requested amounts, but the wrapper discards that capability and can never collect a single side or a specific amount. This discarded flexibility turns into a trap once the way fees accrue and the way the pool retains dust are taken into account.

JavaScript

```
    /// @notice Sweep one pool's protocol fees to `revenueCollector`.
    /// @dev The pool leaves 1 wei per side behind on a full sweep, so
    /// "empty" is a balance of 0 or 1.
    function collectProtocolFees(address pool) public onlyRole(BOT)
    returns (uint128 amount0, uint128 amount1) {
        _requireListaPool(pool);

        (uint128 pending0, uint128 pending1) =
        IListaV3Pool(pool).protocolFees();
        if (pending0 <= 1 && pending1 <= 1) return (0, 0);

        address recipient = revenueCollector;
        (amount0, amount1) =
        IListaV3Pool(pool).collectProtocol(recipient, type(uint128).max,
        type(uint128).max);

        emit ProtocolFeeCollected(pool, recipient, amount0, amount1);
    }
```

### Description

Protocol fees accrue to whichever token is the swap input. A zeroForOne swap books fees to protocolFees.token0, and a oneForZero swap books fees to protocolFees.token1. A pool whose trading flow is predominantly in one direction therefore accumulates fees on only one side while the other side stays flat. Separately, whenever a side is collected the pool leaves exactly 1 wei behind on that side to keep its storage slot warm, so a side that was previously collected sits at exactly 1 until new swaps in that direction occur.

### Impact

Combining these behaviors, a one directional pool reaches the state where the large side collects  $C - 1$  and transfers it, while the quiet side computes  $\text{amount} = \min(\text{max}, 1)$

|                       |                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                       | = 1, then decrements that amount to 0 because amount == protocolFees.token1, and finally performs a transfer of 0 tokens to the recipient. If the quiet side token reverts on zero value transfers, which is a known nonstandard ERC20 behavior, that transfer fails. Because both transfers happen atomically, the already processed large side is rolled back as well. |
| <b>Recommendation</b> | Consider allowing the flexibility of performing amount specific fee collection for each side and requesting each side only when it holds more than 1, for example req0 = pending0 > 1 ? type(uint128).max : 0 and req1 = pending1 > 1 ? type(uint128).max : 0, then call collectProtocol(recipient, req0, req1).                                                         |
| <b>Status</b>         | [Open / Fixed / Acknowledged]                                                                                                                                                                                                                                                                                                                                            |

| [I02] Missing no-op guard in setRevenueCollector() allows a redundant event emission |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Contract</b>                                                                      | ListaV3FactoryOwner.sol                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Severity</b>                                                                      | Informational                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Description</b>                                                                   | <p><b>Description</b><br/>The ListaV3FactoryOwner.setRevenueCollector() function does not verify that the new revenue collector address differs from the one currently stored.</p> <pre> JavaScript function setRevenueCollector(address _revenueCollector) external onlyRole(MANAGER) {     require(_revenueCollector != address(0), 'revenueCollector=0');     emit RevenueCollectorChanged(revenueCollector,     _revenueCollector);     revenueCollector = _revenueCollector; } </pre> <p><b>Impact</b><br/>When a MANAGER calls it with the address that is already set, the function still runs and emits a RevenueCollectorChanged(x, x) event where the previous and current values are identical. This has no effect on funds or contract state and is purely cosmetic, but it can mislead off chain monitoring and log consumers into believing the destination actually changed when it did not.</p> |
| <b>Recommendation</b>                                                                | Consider adding a check that reverts, or returns early, when the supplied address equals the current revenueCollector value,                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Status</b>                                                                        | [Open / Fixed / Acknowledged]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

# Notes

## [N01] MANAGER can redirect the entire fee stream and role assignment is unconstrained at deployment

**Contract** ListaV3FactoryOwner

**Description**

The fee destination is controlled by setRevenueCollector, which is gated only by the MANAGER role and only checks that the new address is not the zero address. A compromised or careless MANAGER can repoint the destination to any nonzero address, after which every future sweep performed by collectProtocolFees and collectProtocolFeesBatch sends funds there.

**Description**

JavaScript

```
function setRevenueCollector(address _revenueCollector) external
onlyRole(MANAGER) {
    require(_revenueCollector != address(0), 'revenueCollector=0');
    emit RevenueCollectorChanged(revenueCollector,
    _revenueCollector);
    revenueCollector = _revenueCollector;
}
```

**Impact**

The full protocol fee stream can be diverted with a single key compromise. The contract custodies no balance, so existing funds are not at risk, but all ongoing revenue is.

## [N02] One way factory ownership transfer can permanently freeze protocol fee administration

**Contract** ListaV3FactoryOwner

**Description**

As noted in the code comments, transferFactoryOwnership calls the factory's setOwner, which is an immediate single step transfer with no acceptance step by the incoming owner. The function validates only that the new owner is not the zero address.

**Description**

JavaScript

```
/// @notice Hand the factory to `newOwner`. One-way and immediate -
the factory's own `setOwner`
/// is single-step. An unreachable address permanently freezes
fee-tier and protocol-fee admin.
function transferFactoryOwnership(address newOwner) external
onlyRole(DEFAULT_ADMIN_ROLE) {
```

```

require(newOwner != address(0), 'newOwner=0');
IListaV3Factory(factory).setOwner(newOwner);
}

```

#### Impact

If DEFAULT\_ADMIN\_ROLE transfers ownership to an address that cannot exercise factory ownership, such as an address with no code or a contract that cannot call the factory, then enabling fee amounts, setting protocol fees, and collecting protocol fees across every pool become permanently unreachable, because the factory provides no recovery path. The result is an irreversible loss of all factory owner powers.

### [N03] Sole DEFAULT\_ADMIN\_ROLE holder can brick the contract by renouncing or revoking its own role

**Contract** ListaV3FactoryOwner

#### Description

The inherited AccessControl functions renounceRole and revokeRole let the only account holding DEFAULT\_ADMIN\_ROLE remove itself, and nothing guards against removing the last admin.

#### Description

#### Impact

If that occurs, the contract permanently loses the ability to perform UUPS upgrades, to use the transferFactoryOwnership escape hatch, and to manage the MANAGER role. The contract and the factory ownership it holds would then be frozen in their current configuration with no way to recover.

### [N04] Strict batch collection lets a single failing pool block collection for all pools

**Contract** ListaV3FactoryOwner

#### Description

collectProtocolFeesBatch iterates the supplied pools and calls collectProtocolFees on each with no error isolation, so any single pool that reverts causes the entire batch to revert.

#### Description

JavaScript

```

/// @dev Strict: a non-pool entry reverts the batch rather than being
skipped, so a bad bot list
/// surfaces instead of silently under-collecting. Empty pools are
no-ops, not failures.
function collectProtocolFeesBatch(address[] calldata pools) external
onlyRole(BOT) {
    for (uint256 i = 0; i < pools.length; i++) {
        collectProtocolFees(pools[i]);
    }
}

```

**Impact**

A pool whose token blacklists the revenueCollector, is paused, returns false on transfer, or otherwise reverts during collectProtocol blocks collection for every other pool in the same call.